Website Streak Tracker — Full Project Documentation

> Built with Next.js 15, TypeScript, Supabase, and Tailwind CSS v4

Table of Contents

---

1. Project Overview

Website Streak Tracker is a web application that allows users to track how consistently they visit specific websites daily over a defined number of days (e.g. 30, 90, 100 days).

Core features:

- GitHub OAuth authentication via Supabase
- Create goals to track specific websites
- Log daily visits with one click
- Automatic streak calculation (resets if a day is missed)
- Dashboard showing streaks, progress, and stats

---

2. Tech Stack

Technology	Version	Purpose
Next.js	15 (latest)	React framework with App Router
TypeScript	5+	Type safety across the entire codebase
Supabase	Latest	PostgreSQL database + Auth
Tailwind CSS	v4	Utility-first styling
@supabase/ssr	Latest	Cookie-based auth for Next.js App Router

Why these choices:

- Next.js App Router enables server components, server actions, and file-based routing
- Supabase replaces a traditional backend — it provides auth, database, and row-level security out of the box
- Tailwind v4 removes the need for a config file — everything lives in globals.css
- `@supabase/ssr` is the correct modern package for Supabase in Next.js (replaces the deprecated `@supabase/auth-helpers-nextjs`)

---

3. Folder Structure

```
website-streak-tracker/
├── app/
│   ├── auth/
│   │   └── callback/
│   │       └── route.ts          # Handles GitHub OAuth redirect
│   ├── dashboard/
│   │   ├── actions.ts            # Server actions: createGoal, logVisit, deleteGoal
│   │   ├── error.tsx             # Error boundary for dashboard
│   │   ├── loading.tsx           # Loading state for dashboard
│   │   └── page.tsx              # Main dashboard page (server component)
│   ├── login/
│   │   ├── actions.ts            # Server actions: signInWithGitHub, signOut
│   │   └── page.tsx              # Login page UI
│   ├── globals.css               # Tailwind v4 import
│   ├── layout.tsx                # Root layout
```

```
│   ├── not-found.tsx          # 404 page
│   └── page.tsx               # Home page (redirects based on auth)
├── components/
│   ├── EmptyState.tsx         # Shown when user has no goals
│   ├── GoalCard.tsx           # Individual goal card with stats
│   ├── GoalForm.tsx           # Form to create a new goal
│   ├── LogVisitButton.tsx     # Button to log today's visit
│   └── StatsBar.tsx           # Summary stats across all goals
├── types/
│   └── database.ts            # TypeScript types matching DB tables
├── utils/
│   └── supabase/
│       ├── client.ts          # Supabase client for client components
│       ├── server.ts          # Supabase client for server components
│       └── session.ts         # Session refresh + route protection
├── proxy.ts                   # Next.js 15 proxy (replaces middleware.ts)
├── .env.local                 # Environment variables (never commit this)
├── next.config.ts             # Next.js configuration
├── tsconfig.json              # TypeScript configuration
└── package.json               # Dependencies
```

---

4. Phase 1 — Project Setup

What we did

Initialized the project and set up all the foundational tooling.

Key decisions and explanations

`create-next-app` flags used:

- TypeScript: yes — type safety prevents bugs at compile time
- App Router: yes — the modern Next.js routing system (replaces the old Pages Router)
- Tailwind CSS: yes — utility classes mean no separate CSS files needed
- Turbopack: no — still experimental, standard webpack is more stable

Tailwind v4 change: In Tailwind v4 (installed by default with Next.js 15), the `tailwind.config.ts` file no longer exists. Configuration moved into `globals.css` using `@import "tailwindcss"`. All utility classes work identically.

Three Supabase client files: Supabase requires different client instances depending on where the code runs in Next.js:

- `utils/supabase/client.ts` — used inside Client Components (`'use client'`). Uses `createBrowserClient` which reads cookies from the browser.
- `utils/supabase/server.ts` — used inside Server Components and Server Actions. Uses `createServerClient` which reads cookies from the Next.js cookie store.
- `utils/supabase/session.ts` — used inside the proxy (middleware). Refreshes the user session on every request so it never expires silently.

Why `proxy.ts` instead of `middleware.ts`: Next.js 15 renamed the middleware file convention from `middleware.ts` to `proxy.ts`. The exported function name also changed from `middleware` to `proxy`. The purpose is identical — it runs on every request before the page renders.

`.env.local`: Stores secret configuration that should never be committed to Git. Next.js automatically loads this file. Variables prefixed with `NEXT_PUBLIC_` are exposed to the browser; variables without the prefix are server-only.

---

5. Phase 2 — Authentication

What we built

GitHub OAuth login flow using Supabase Auth.

How the auth flow works

```
User clicks "Continue with GitHub"
  → signInWithGitHub() server action runs
  → Supabase generates a GitHub OAuth URL
  → User is redirected to GitHub
  → User authorizes the app on GitHub
  → GitHub redirects to /auth/callback?code=xxxxx
  → route.ts exchanges the code for a Supabase session
  → Session is stored in cookies
  → User is redirected to /dashboard
```

File explanations

`app/login/page.tsx` A simple server component that renders the login UI. Uses an HTML `<form>` with a server action — this means it works even without JavaScript enabled.

`app/login/actions.ts` Contains two server actions marked with `'use server'`:

- `signInWithGitHub()` — calls `supabase.auth.signInWithOAuth()` which generates a GitHub OAuth URL, then redirects the user there

- `signOut()` — calls `supabase.auth.signOut()` which clears the session cookie, then redirects to `/login`

Server actions are functions that run on the server but can be called from the client. They replace traditional API routes for form submissions.

`app/auth/callback/route.ts` A Next.js Route Handler (API endpoint). GitHub redirects here after authorization with a `code` parameter. The code is exchanged for a real session using `supabase.auth.exchangeCodeForSession(code)`. This is the standard OAuth PKCE flow.

`utils/supabase/session.ts` Runs on every request via the proxy. It calls `supabase.auth.getUser()` which refreshes the session token if it's about to expire. Without this, users get logged out randomly. It also redirects unauthenticated users to `/login` if they try to access protected routes.

Route protection logic:

```typescript
if (
  !user &&
  !request.nextUrl.pathname.startsWith('/login') &&
  !request.nextUrl.pathname.startsWith('/auth')
) {
  redirect('/login')
}
```

Any route that isn't `/login` or `/auth/*` requires authentication. The dashboard, home page, and any future pages are automatically protected.

Important config

In Supabase dashboard, GitHub OAuth requires:

- A GitHub OAuth App with the callback URL set to `https://your-project.supabase.co/auth/v1/callback`
- The Client ID and Client Secret from GitHub pasted into the Supabase GitHub provider settings

---

6. Phase 3 — Database Design

Tables created

`public.goals` Stores each tracking goal a user creates.

Column	Type	Notes
id	uuid	Primary key, auto-generated with `gen_random_uuid()`
user_id	uuid	Foreign key → `auth.users(id)`. Links goal to its owner
website_url	text	The URL being tracked e.g. `https://github.com`
website_name	text	Friendly display name e.g. `GitHub`
target_days	integer	Goal length — 7, 14, 30, 60, 90, or 100 days
created_at	timestamptz	When the goal was created, stored in UTC

`public.visit_logs` Records each daily visit to a tracked website.

Column	Type	Notes
id	uuid	Primary key, auto-generated
goal_id	uuid	Foreign key → `goals(id)`. Links log to its goal
user_id	uuid	Foreign key → `auth.users(id)`. Stored here for faster RLS checks
visited_at	date	Date only (no time). Prevents timezone confusion
created_at	timestamptz	Exact timestamp of when the log was created

Unique constraint on `visit_logs`:

```sql
unique(goal_id, visited_at)
```

This is a database-level rule that prevents logging the same goal twice on the same day. Even if the application code has a bug, the database will reject duplicate entries.

Relationships

```
auth.users (1) ──── (many) goals
goals      (1) ──── (many) visit_logs
```

- One user can have many goals
- One goal can have many visit logs (one per day)
- When a user is deleted, all their goals are deleted (`on delete cascade`)
- When a goal is deleted, all its visit logs are deleted (`on delete cascade`)

Row Level Security (RLS)

RLS is a PostgreSQL feature that enforces data access rules at the database level. Even if application code has a bug that tries to fetch another user's data, the database will block it.

Every table has RLS enabled with policies like:

```sql
-- Users can only see their own goals
create policy "Users can view own goals"
  on public.goals for select
  using (auth.uid() = user_id);
```

`auth.uid()` is a Supabase function that returns the ID of the currently authenticated user.

Why `visited_at` is `date` not `timestamp`

Using `date` (e.g. `2025-01-15`) instead of `timestamp` (e.g. `2025-01-15 14:32:00+00`) means:

- No timezone conversion issues
- Simple equality checks (`visited_at = current_date`)
- The unique constraint works correctly regardless of what time of day the visit is logged

---

7. Phase 4 — Core Features

TypeScript Types (`types/database.ts`)

Three types defined to match the database exactly:

- `Goal` — mirrors the `goals` table columns

- `VisitLog` — mirrors the `visit_logs` table columns
- `GoalWithStats` — extends `Goal` with computed fields: `currentStreak`, `totalVisits`, `lastVisitDate`, `visitedToday`

The `GoalWithStats` type uses TypeScript intersection (`Goal &`) to add computed fields without duplicating the base type.

Streak Calculation (`utils/streak.ts`)

Three exported functions:

`calculateStreak(visitLogs)` The core algorithm:

1. Extract all `visited_at` dates and sort them newest first
2. Remove duplicates (safety measure)
3. Check if the most recent visit was today or yesterday — if not, streak is 0
4. Count consecutive days going backwards until a gap is found
5. Return the count

The reason we allow yesterday: if it's 9am and you haven't logged today yet, your streak shouldn't show as broken. You still have the rest of the day.

`hasVisitedToday(visitLogs)` Compares today's date (`new Date().toISOString().split('T')[0]`) against all `visited_at` values. Returns `true` if any match.

`getLastVisitDate(visitLogs)` Sorts all visit dates and returns the most recent one, or `null` if there are no visits.

Server Actions (`app/dashboard/actions.ts`)

All three functions are marked `'use server'` and run on the server:

`createGoal(formData)`

1. Verifies the user is authenticated
2. Extracts `website_url`, `website_name`, `target_days` from the form
3. Inserts a new row into `goals`
4. Calls `revalidatePath('/dashboard')` to refresh the page data

`logVisit(goalId)`

1. Verifies the user is authenticated
2. Gets today's date as a string (`YYYY-MM-DD`)

3. Inserts a new row into `visit_logs`
4. Handles error code `23505` (PostgreSQL unique violation) gracefully — this means the user already logged today
5. Calls `revalidatePath('/dashboard')` to refresh the page data

`deleteGoal(goalId)`

1. Verifies the user is authenticated
2. Deletes the goal matching both `id` AND `user_id` — the `user_id` check is a second layer of security on top of RLS
3. Because of `on delete cascade`, all visit logs for that goal are also deleted automatically

`revalidatePath('/dashboard')` This is a Next.js function that tells the server to regenerate the cached dashboard page data. Without it, the UI wouldn't update after creating or deleting a goal.

Components

`GoalForm.tsx` — marked `'use client'` because it uses `useState` for open/close toggle and loading state. Uses an HTML form with a server action passed as the `action` prop.

`LogVisitButton.tsx` — marked `'use client'` because it manages local state (loading, done, error). Calls the `logVisit` server action directly on click.

---

8. Phase 5 — Dashboard UI

Architecture

The dashboard page (`app/dashboard/page.tsx`) is a **Server Component**. This means:

- It runs on the server, not in the browser
- It can directly call Supabase without an API layer
- It fetches all data before sending HTML to the browser (fast initial load)
- It cannot use `useState`, `useEffect`, or browser APIs

Client components (`GoalCard`, `GoalForm`, `LogVisitButton`) are embedded inside the server component for interactive parts.

Data fetching strategy

Two parallel database queries:

1. Fetch all goals for the user
2. Fetch all visit logs for the user

Then combine them in JavaScript:

```typescript
const goalsWithStats = goals.map(goal => {
  const goalLogs = visitLogs.filter(log => log.goal_id === goal.id)
  return {
    ...goal,
    currentStreak: calculateStreak(goalLogs),
    totalVisits: goalLogs.length,
    lastVisitDate: getLastVisitDate(goalLogs),
    visitedToday: hasVisitedToday(goalLogs),
  }
})
```

This avoids N+1 queries (fetching logs separately for each goal).

Component breakdown

`StatsBar.tsx` — server-compatible, receives `GoalWithStats[]` and computes summary numbers: total goals, active streaks (streak > 0), visited today, longest streak across all goals.

`GoalCard.tsx` — displays one goal with all its stats. The delete button uses `confirm()` (browser dialog) so it must be a client component.

`EmptyState.tsx` — pure presentational component shown when the user has no goals yet.

`LogVisitButton.tsx` — manages its own visited state locally. When `logVisit` succeeds, it flips to the "Visited today" state without needing a full page reload.

---

9. Phase 6 — Code Quality

Files added

`app/page.tsx` (updated) The home page now checks auth and redirects appropriately — logged in users go to `/dashboard`, logged out users go to `/login`. Previously it just showed static text.

`app/dashboard/loading.tsx` Next.js automatically shows this component while the dashboard server component is fetching data. Uses the special filename `loading.tsx` — no imports needed.

`app/dashboard/error.tsx` Next.js automatically shows this component if the dashboard throws an error. Must be a client component (`'use client'`) because it uses the `reset` function to retry. Uses the special filename `error.tsx`.

`app/not-found.tsx` Shown automatically by Next.js for any URL that doesn't match a route. Uses the special filename `not-found.tsx`.

File naming conventions in Next.js App Router

Filename	Purpose
`page.tsx`	The UI for a route
`layout.tsx`	Wraps pages — persists across navigation
`loading.tsx`	Shown while page data loads
`error.tsx`	Shown if the page throws an error
`not-found.tsx`	Shown for 404s
`route.ts`	API endpoint (no UI)
`actions.ts`	Convention for server actions (not a Next.js special file)

---

10. Key Concepts Explained

Server Components vs Client Components

	Server Component	Client Component
Runs on	Server only	Browser (and server for hydration)
Can use	Database, file system, secrets	useState, useEffect, browser APIs
Marked with	Nothing (default)	`'use client'` at top of file
Can fetch data	Yes, directly	Yes, but via API calls
Examples	`dashboard/page.tsx`	`GoalForm.tsx`, `LogVisitButton.tsx`

Server Actions

Functions marked with `'use server'` that run on the server but can be called from client components. They replace traditional REST API routes for form submissions and mutations. Next.js automatically creates a secure POST endpoint for each server action.

Row Level Security (RLS)

A PostgreSQL feature that filters data at the database level based on the authenticated user. Even if you write a query with no `where` clause, users only see their own data. It's a safety net on top of application-level checks.

The OAuth Flow

OAuth is a protocol that lets users log in with a third-party account (GitHub) without sharing their password with your app. The flow:

1. Your app sends the user to GitHub with a request
2. GitHub asks the user to authorize your app
3. GitHub sends the user back to your app with a temporary code
4. Your app exchanges the code for an access token with GitHub (via Supabase)
5. Supabase creates a session and stores it in cookies

UUID

Universally Unique Identifier — a randomly generated 128-bit ID like `550e8400-e29b-41d4-a716-446655440000`. Used as primary keys instead of sequential integers (1, 2, 3) because they are globally unique and don't expose record counts.

`revalidatePath`

Next.js caches server component data for performance. `revalidatePath('/dashboard')` tells Next.js to throw away the cached version of `/dashboard` and re-fetch fresh data on the next request. Called after any mutation (create, delete, log visit).

---

11. Common Errors & Fixes

`No API key found in request`

Cause: `.env.local` has wrong values or placeholders. **Fix:** Go to Supabase → Settings → API. Copy the Project URL (no trailing path) and the `anon public` key (starts with `eyJ`). Restart the dev server after editing `.env.local`.

`middleware.ts` / `proxy.ts` naming errors

Cause: Next.js 15 renamed `middleware.ts` to `proxy.ts` and the exported function from `middleware` to `proxy`. **Fix:** File must be named `proxy.ts` at project root. Export must be `export async function proxy(...)`.

`Export X doesn't exist in target module`

Cause: The file exists but the function isn't exported, or the file is named `middleware.ts` which conflicts with Next.js internals. **Fix:** Rename utility files away from reserved names. Ensure all functions have the `export` keyword.

`Cannot find module '@/components/X'`

Cause: File has wrong extension (`.ts` instead of `.tsx`), is in wrong folder, or has a syntax error preventing TypeScript from parsing it. **Fix:** Check extension (UI files need `.tsx`), verify folder location, check for syntax errors inside the file.

`selected` prop warning on `<option>`

Cause: React doesn't support the HTML `selected` attribute on `<option>` elements. **Fix:** Use `defaultValue="30"` on the `<select>` element instead.

Streak showing 0 after logging a visit

Cause: Usually a timezone mismatch between the server and the date being stored. **Fix:** The `visited_at` field stores date only (`YYYY-MM-DD`). The streak calculator uses `new Date().toISOString().split('T')[0]` to get today's date in UTC. Both must use the same timezone reference.

---

12. Environment Variables

`.env.local`

```env
NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
NEXT_PUBLIC_SITE_URL=http://localhost:3000
```

Variable	Where to find it	Safe to expose?
`NEXT_PUBLIC_SUPABASE_URL`	Supabase → Settings → API → Project URL	Yes
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase → Settings → API → anon public key	Yes (protected by RLS)
`NEXT_PUBLIC_SITE_URL`	Your own domain / localhost	Yes

For production (Vercel): Add these same variables in Vercel → Project → Settings → Environment Variables. Change `NEXT_PUBLIC_SITE_URL` to your production domain e.g. `https://your-app.vercel.app`.

Summary

This project demonstrates a production-ready full-stack Next.js application with:

- Server-side rendering with server components
- Secure authentication with OAuth
- Database with row-level security
- Real-time UI updates via server actions and `revalidatePath`
- TypeScript throughout for type safety
- Clean component architecture separating concerns

The patterns used here — server components, server actions, Supabase SSR, RLS — are the current best practices for Next.js + Supabase applications as of 2025.